Restricciones de Integridad declarativas

Restricciones de Integridad (RI)

- Condiciones que restringen los posibles valores válidos de los datos en la BD
- Actividad importante del diseño de una BD: identificar e incorporar las RI que deben cumplirse



- El SGBD controla las operaciones sobre los datos para garantizar el cumplimiento de las RI:
  - → **rechazando** operaciones (altas, bajas, modificaciones), o
  - → realizando **acciones reparadoras** sobre los datos

Ambas respuestas deben dejar la BD en un **estado consistente**

Clasificación de RI

Según su naturaleza

- **RI inherentes (implícitas):**
  propias del modelo relacional, no necesitan declararse

- **RI explícitas:**
  se especifican en el esquema de la BD mediante sentencias del DDL

  Ej: restricciones de dominio, clave, nulidad, integridad referencial

- **RI semánticas (reglas del negocio):**
  restricciones adicionales según la lógica de la organización
  (planteadas en forma declarativa o procedural → *triggers*)

Clasificación de RI

Según estados involucrados

- **RI de estado:** restringen los valores de los datos **en un momento dado**
  - **Nulidad:** el valor de un atributo puede/no puede ser nulo
  - **Dominio:** valores permitidos para un atributo
  - **Cardinalidad:** cantidad de participaciones de una entidad en una relación
  - **Participación:** puede ser obligatoria u opcional

- **RI de transición:** controlan **cambios entre estados sucesivos**
  - Requieren conocer el estado "anterior" y "nuevo"
  - No pueden plantearse en forma declarativa sino procedural (*triggers*)

RI Referencial (RIRs)

- Condiciones sobre los valores de las claves extranjeras - *foreign keys* (FK)
- El conjunto de valores de una **FK** en una relación ***R*** debe existir como clave primaria (o alternativa) en una relación ***R´*** a la que la FK hace referencia, o ser nulos

si es posible (por la definición de los datos)

***R* [*lista_col_referencia*] << *R´* [*lista_col_referenciada*] : (b,m)**

b= acción referencial ante baja

m= acción ref. ante modificación (a derecha)

- ***R*** y ***R´*** podrían coincidir
- *lista_col_referencia* y *lista_col_referenciada* deben tener igual número de columnas y tipos de datos compatibles (no requiere igualdad de nombres)

RI Referencial (RIRs)

```
CREATE TABLE NombreTabla
   ( { nombre_columna  TipoDato  [NOT NULL] ... ,}
     [ [CONSTRAINT PK_nom] PRIMARY KEY (lista_columnasPK),]
     { [ [CONSTRAINT U_nom] UNIQUE (lista_columnas),] }
     { [ [CONSTRAINT FK_nom] FOREIGN KEY (lista_col_referencia)
              REFERENCES nombreTablaRef [(lista_col_referenciada)]
              [ MATCH {FULL | PARTIAL | SIMPLE} ]
              [ON UPDATE AccionRef]
              [ON DELETE AccionRef] ] } ....
   );
```

comportamiento ante borrado o modificación de datos referenciados

```
ALTER TABLE NombreTabla
   ADD [CONSTRAINT FK_nom] FOREIGN KEY (lista_col_referencia) ... ;
```

Diferentes acciones referenciales:
AccionRef = **NO ACTION** | **CASCADE** | **SET NULL** | **SET DEFAULT** | **RESTRICT**

Acciones Referenciales (delete)

R [*lista_col_referencia*] << *R'* [*lista_col_referenciada*]: (**b**, m)

¿Qué sucede al intentar BORRAR una tupla referenciada en *R'*?

→ *Rechaza la operación*



→ **NO ACTION:** impide borrar la tupla referenciada en *R'* si hay referencias en *R* *(opción por defecto en SQL estándar y PostgreSQL)*

→ **RESTRICT:** misma semántica pero se chequea antes de otras RI

→ *Procede y realiza acciones reparadoras adicionales*



→ **CASCADE:** borra la tupla referenciada en R' y propaga el borrado a las tuplas en R que la referenciaban mediante la FK

→ **SET NULL:** borra la tupla referenciada en R' y las tuplas que la referenciaban en R ponen a nulo la FK (si admite nulos)

→ **SET DEFAULT:** borra la tupla referenciada en R' y las tuplas que la referenciaban ponen en la FK el valor por defecto definido para la misma

Acciones Referenciales (update)

R [*lista_col_referencia*] << *R´* [*lista_col_referenciada*]: (b, **m**)

¿Qué sucede si se intenta MODIFICAR el valor de la clave de la tupla referenciada en *R´*? (modificación a derecha)

→ ***Rechaza* la operación**



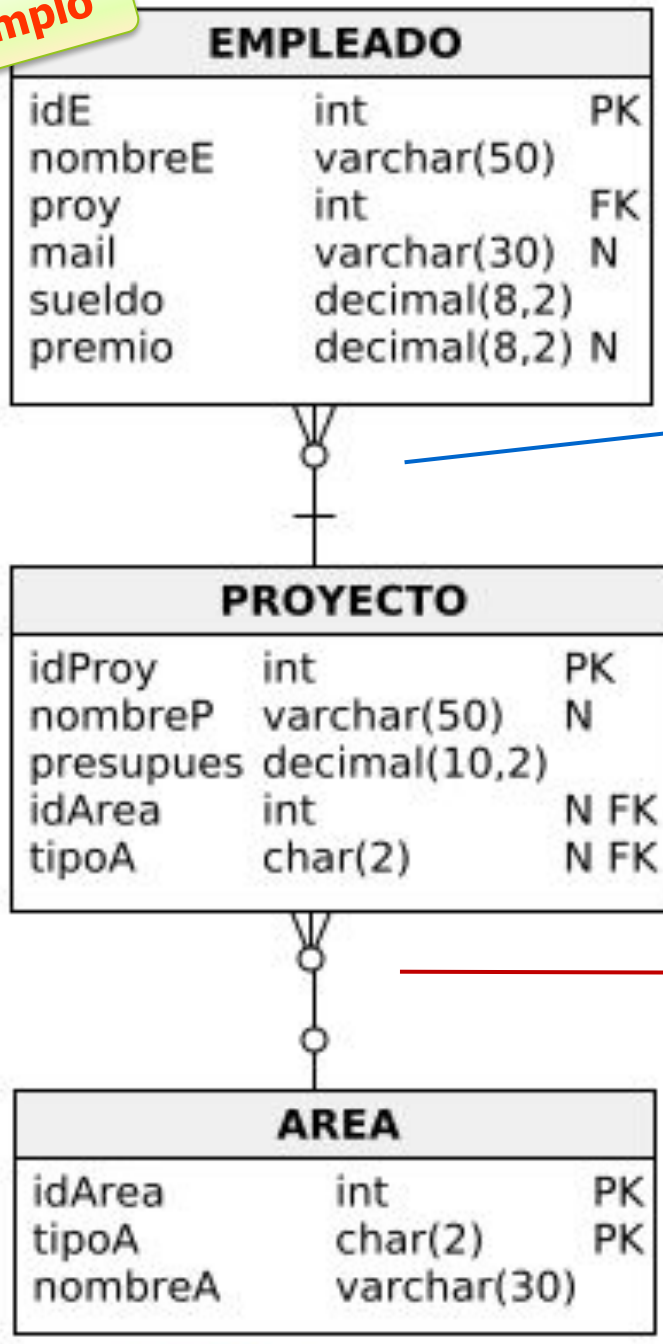
- → **NO ACTION:** no permite modificar el valor de la clave en la tupla referenciada en *R´* si hay referencias en *R* *(opción por defecto)*
- → **RESTRICT:** misma semántica pero se chequea antes de otras RI

→ ***Procede y realiza acciones reparadoras adicionales***



- ✓ **CASCADE:** modifica el valor de la clave de la tupla referenciada en *R´* y propaga en R la modificación a las tuplas que la referenciaban
- ✓ **SET NULL:** modifica la tupla referenciada en *R´* y las tuplas que la referenciaban en *R* ponen a nulo la FK (si admite nulos)
- ✓ **SET DEFAULT:** modifica la tupla referenciada en *R´* y las tuplas que la referenciaban en *R* ponen en la FK el valor por defecto definido

Ejemplo




```
ALTER TABLE EMPLEADO
ADD CONSTRAINT fk_empleado_proyecto
   FOREIGN KEY (proy)
   REFERENCES PROYECTO (idProy)
   ON UPDATE CASCADE;
```

ON DELETE: *al no especificar, la opción por defecto es* NO ACTION

```
ALTER TABLE PROYECTO
ADD CONSTRAINT fk_proyecto_area
   FOREIGN KEY (idArea, tipoA)
   REFERENCES area (idArea, tipoA)
   ON DELETE CASCADE
   ON UPDATE CASCADE;
```

¿Qué sucedería ante un delete en AREA?

Ejemplo

EMPLEADO (proy) << PROYECTO (idProy)

¿Cómo proceden las operaciones según las diferentes acciones referenciales?

Nota: Considere la instancia dada y resultados individuales, no acumulativos

FK → PK

EMPLEADO			
idE	nombreE	…	**proy**
1	emp1	…	101
2	emp2	…	101

PROYECTO		
idProy	nombreP	…
101	proy101	…
102	proy102	…

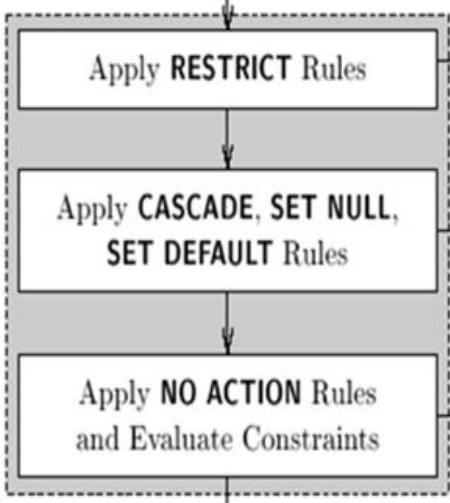
1. UPDATE proyecto set IdProy = 201 where idProy = 101;
2. UPDATE proyecto set IdProy = 202 where nombreP = ‘proy102’;
3. UPDATE empleado set proy=102 WHERE idE= 1;
4. UPDATE empleado set proy=100 WHERE idE= 2;
5. DELETE FROM proyecto WHERE idProy = 101;
6. DELETE FROM proyecto WHERE idProy = 102;
7. DELETE FROM proyecto WHERE idProy > 100;

Importante: *una operación se ejecuta completamente o se rechaza (no hay efectos parciales)*



Diferencia RESTRICT / NO ACTION

Misma semántica, varía el momento de su verificación



Modelo de ejecución (parcial) según SQL estándar

PELICULA			
id_pel	**titulo**	**id_prod**	**id_dir**
p1	tit1	1	3
p2	tit2	1	2
p3	tit3	2	2

STAFF	
id_staff	**nombre**
1	Juan
2	Pablo
3	Maria

DELETE FROM STAFF
WHERE id_staff = 2;

R1: PELICULA (id_prod) << STAFF (id_staff) delete i) **RESTRICT**, ii) **NO ACTION**
R2: PELICULA (id_dir) << STAFF (id_staff) delete **CASCADE**

i) RESTRICT

- R1 se verifica antes que R2
- Se detecta la referencia a STAFF(2) en id_prod y se **rechaza la operación**
- Las tablas no se modifican

ii) NO ACTION

- R2 se verifica antes que R1
- Se **eliminan** en cascada las películas cuyos id_dir referencian a STAFF(2)
- Al no haber referencias en id_prod, se **elimina** STAFF(2)

RIRs – Tipos de Matching

R [*lista_col_referencia*] << ***R'*** [*lista_col_referenciada*]

```
CREATE TABLE NombreTabla
  ( { nombre_columna  TipoDato  [NOT NULL] ... ,}
    [ [CONSTRAINT PK_nom] PRIMARY KEY (lista_columnasPK),]
    { [ [CONSTRAINT U_nom] UNIQUE (lista_columnas),] }
    { [ [CONSTRAINT FK_nom] FOREIGN KEY (lista_col_referencia)
          REFERENCES nombreTablaRef [(lista_col_referenciada)]
          [ MATCH {FULL | PARTIAL | SIMPLE} ]
          [ON UPDATE AccionRef]
          [ON DELETE AccionRef] ] } .....
  );
```

comportamiento ante presencia de nulos en la FK

RIRs – Tipos de Matching

R [*lista_col_referencia*] << ***R'*** [*lista_col_referenciada*]

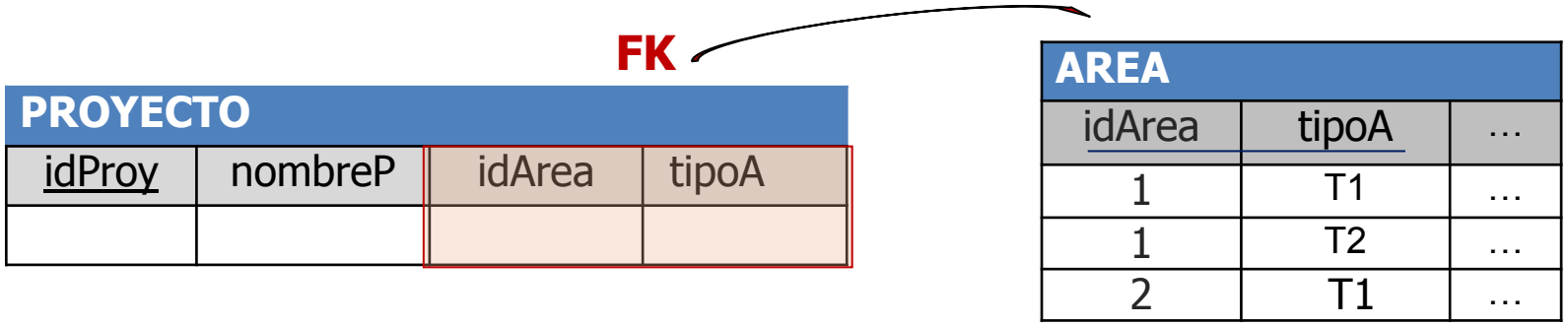
Una RIR se satisface si para cada tupla en la tabla R se verifica que:

Ninguna de las columnas de la FK en *R* es NULL y existe una tupla en la tabla referenciada *R'* cuyos valores de clave coinciden con los de tales columnas, o ...

- alguna de las columnas en la FK en *R* es NULL (**MATCH SIMPLE**)
- los valores no nulos de la FK en *R* se corresponden con los respectivos valores de la clave en al menos una tupla de *R'* (**MATCH PARTIAL**)
- todos los valores de la FK en *R* son NULL (**MATCH FULL**)

Tiene sentido especificar la cláusula matching si la FK es compuesta y sus atributos admite/n nulos

Ejemplo:



Las siguientes altas en Proyecto ¿serían aceptadas o rechazadas?

(considerando que los atributos de la FK admiten nulos)

insert into PROYECTO (idProy, nombreP, idArea, tipoA)

… values (1, 'proy1', 1, 'T1');

… values (2, 'proy2', 2, 'T2');

… values (3, 'proy3', null, null);

… values (4, 'proy4', null, 'T1');

… values (5, 'proy5', 3, null);

MATCHING

Simple	Parcial	Full
√	√	√
x	x	x
√	√	√
√	√	x
√	x	x

Restricciones de Integridad
Declarativas según SQL estándar

- Restricciones de integridad (RI) específicas sobre los datos
  → **reglas de negocio**
- El estándar SQL permite la **especificación declarativa** de RI de distinto tipo, según la estructura jerárquica del modelo relacional:

→ **RI de atributo o dominio (DOMAIN)**

→ **RI de tabla asociada a uno o más atributos (CHECK de tupla)**

→ **RI de tabla asociada a varias tuplas (CHECK de tabla)**

→ **RI generales de la base de datos (ASSERTION)**

Restricciones de Integridad
Declarativas

- El SGBD permite definir las RI y controla su cumplimiento en todo momento
- Se **activan** ante alguna operación sobre los objetos afectados y su incumplimiento genera el **rechazo** de la operación
- Una RI se cumple si la condición a chequear **no evalúa falso**, sino verdadero o desconocido (ante presencia de nulos)
- Al especificar una RI declarativa, el SGBD la verifica sobre los datos existentes en la BD → **si no se satisface, rechaza su definición**

RI de Dominio/Atributo

- Permite definir el conjunto de los valores válidos de un atributo
- Ámbito: **atributo**

Se puede especificar en la sentencia CREATE TABLE

```
CREATE TABLE NomTabla (
   ...
  NomAtrib  TipoDato [ DEFAULT ValorDefecto ]
  [ [CONSTRAINT NomRestriccion] CHECK (condición)]...);
```

o definirla en un dominio (útil para abstraer propiedades comunes) y declarar cada atributo asociado al dominio

```
CREATE DOMAIN  NomDominio
  AS TipoDato  [ DEFAULT ValorDefecto ]
  [ [CONSTRAINT NomRestriccion] CHECK (condición)];

CREATE TABLE NomTabla (
   ...
  NomAtrib NomDominio ... );
```

RI de Dominio/Atributo

Ejemplo

El sueldo de un empleado es un valor mayor a 0 y con 2 decimales

EMPLEADO (idE, ..., sueldo)

al crear la tabla:

```
CREATE TABLE Empleado
( ...
  sueldo decimal (10,2)
     CONSTRAINT SueldoPos CHECK (sueldo > 0),
  ... );
```

o definiendo un dominio:

```
CREATE DOMAIN SueldoPos
AS Numeric (10,2) CHECK (value > 0);
```

```
CREATE TABLE Empleado
(  ...
   sueldo SueldoPos,
   ...  );
```

RI (check) de Tupla

- Es una restricción específica sobre los valores que puede tomar una combinación de atributos en una tupla
- Ámbito: **tupla** (la RI se comprueba para cada fila que se inserta o modifica en la tabla)

```
CREATE TABLE NombreTabla
(  …
  { [ [CONSTRAINT NomRestriccion] CHECK (condición) ] }
… );
```

o:

```
ALTER TABLE NombreTabla
  ADD [CONSTRAINT NomRestriccion] CHECK (condición) ;
```

RI (check) de Tupla

Ejemplo

Salvo para empleados del proyecto 101, el premio asignado a un empleado no puede superar el 25% de su sueldo

EMPLEADO(idE, ..., proy, sueldo, premio)

```
ALTER TABLE Empleado
ADD CONSTRAINT chk_premio
 CHECK ( proy = 101 OR
         premio <= 0.25*sueldo );
```

RI (check) de Tabla

- Restricción que afecta diferentes tuplas de una misma tabla
- Ámbito: **tabla** (la RI se comprueba cada vez que se actualiza la tabla)
- Se define de manera similar al check de Tupla

```
CREATE TABLE NombreTabla
(  …..
  { [ [CONSTRAINT NomRestriccion] CHECK (condición) ] }
 … );

o:
ALTER TABLE NombreTabla
   ADD [CONSTRAINT NomRestriccion] CHECK (condición) ;
```

RI (check) de Tabla

Ejemplo

En cada proyecto pueden trabajar hasta 10 empleados

EMPLEADO (idEmp, nombreEmp, proy, mail, sueldo, premio)

SQL no permite expresar directamente
"todo X debe cumplir C"

→ se utiliza su equivalente:
"No existe X que no cumpla C"

```
ALTER TABLE Empleado
ADD CONSTRAINT chk_cant_empl
CHECK ( NOT EXISTS
        (SELECT proy FROM Empleado
         GROUP BY proy
         HAVING COUNT(*) > 10 )
    );
```

proyectos con más de 10 empleados → **NO cumplen** la condición

Si la consulta devuelve una tabla vacía → NOT EXISTS es Verdadero y el CHECK se verifica

RI Generales (ASSERTIONS)

- Son restricciones sobre un número arbitrario de atributos de 2 o más tablas
- No están asociadas a un elemento (tabla o dominio) en particular
- Ámbito: **base de datos**
- Sintaxis (en SQL estándar):

CREATE ASSERTION NomAssertion CHECK (condición);

- Se activaría ante actualizaciones sobre cualquiera de los objetos involucrados → alto costo para comprobación y mantenimiento (no implementada en los DBMS)

RI Generales (ASSERTIONS)

Ejemplo

El sueldo de cada empleado de un proyecto no puede superar el presupuesto del proyecto en que trabaja

EMPLEADO (idEmp, …, sueldo, proy,…)

PROYECTO (IdProy, …, presupuesto)

La RI afecta 2 tablas: se debe comparar el sueldo de cada empleado contra el presupuesto del proyecto en que trabaja

```
CREATE ASSERTION sueldo_menor_presup
  CHECK ( NOT EXISTS ( SELECT 1 FROM Empleado e
                       JOIN Proyecto p ON e.proy = p.IdProy
                       WHERE  e.sueldo > p.presupuesto
 ));
```

empleados con sueldo mayor al presupuesto del proyecto (no cumplen la condición)

Incorporación de restricciones – estándar SQL

```
CREATE TABLE NombreTabla
    ( { nombre_columna  TipoDato | nomDom [NOT NULL] [DEFAULT valorDef]
                    [ [CONSTRAINT nom_restr] CHECK (condicion)]  ... ,}
      [ [CONSTRAINT nom_restr_PK] PRIMARY KEY (lista_columnasPK),]
      { [ [CONSTRAINT nom_restr_U] UNIQUE (lista_columnas),] }
      { [ [CONSTRAINT nom_restr_FK] FOREIGN KEY (lista_col_referencia)
                REFERENCES nombreTablaRef [(lista_col_referenciada)]
                [ MATCH {FULL | PARTIAL | SIMPLE} ]
                [ ON UPDATE AccionRef]  [ ON DELETE AccionRef] ], }
      { [ [CONSTRAINT nom_restr ] CHECK (condicion),] }
    );
```

Nota:
[...] opcional
{...} uno o más
| posibilidades

```
CREATE DOMAIN NomDominio AS TipoDato [ DEFAULT ValorDefecto ]
  [ [CONSTRAINT NomRestriccion] CHECK (condicion)];
```

```
ALTER TABLE NombreTabla
      ADD [ CONSTRAINT nom-restr ] restriccion  ;

      donde restriccion  =  PRIMARY KEY (lista_columnasPK) |
                            UNIQUE (lista_columnas) |
                            FOREIGN KEY (lista_col_referencia) .... |
                            CHECK (condicion)
```

```
CREATE ASSERTION  NomAssertion CHECK (condicion);
```

Consideraciones

- Determinar el **ámbito** de la RI: atributo, tupla, tabla o general?
- Identificar el recurso declarativo **más restrictivo**, según:
  - el estándar SQL
  - las posibilidades del SGBD
- Los nombres de las RI deben ser **únicos** dentro del esquema
- El SGBD verifica las RI afectadas por cada operación:
  - si alguna no se cumple, rechaza la operación (sin dejar efectos parciales)
  - la BD debe permanecer en un estado consistente
- Opciones de tiempo de verificación de RI:
  - *deferrable*: permite diferir la comprobación de la restricción
  - *not deferrable*: se comprueba tras cada operación de actualización individual dentro de una transacción

Alternativa: SQL procedural
cuando no es posible de modo declarativo

Los SGBD no siempre brindan recursos declarativos para:

- implementar restricciones de tabla complejas
- establecer restricciones generales (assertions)
- definir ciertas acciones referenciales o de matching
- otros requerimientos…

SQL procedural: combina sentencias SQL y estructuras de control para implementar reglas y acciones específicas

Lenguaje PL/SQL permite implementar:

- **Triggers:** código que se **dispara automáticamente ante un evento y ejecuta la/s acción/es** establecidas (sentencias SQL + sentencias de control)
- **Stored procedures:** código almacenado que se invoca explícitamente
- **Functions:** predefinidas o definidas por el usuario, pueden ser invocadas desde un trigger, stored procedure o explícitamente

BIBLIOGRAFÍA

Date, C., "An Introduction to Database Systems". 7º ed., Addison Wesley, 2000

Elmasri, R., Navathe, S., "Fundamentals of Database Systems", Addison Wesley, 2011 (y 2016)

Silberschatz, A., Korth, H, Sudarshan, S., "Database System Concepts", McGraw Hill, 2002

Sumathi S., Esakkirajan S., Fundamentals of Relational Database Management Systems, 2007

Documentación PostgreSQL C.T.- Constraints 5.5

